

Demonstration of VRSight: AI-Driven Real-Time Descriptions to Enhance VR Accessibility for Blind People

Daniel Killough
Department of Computer
Sciences
University of
Wisconsin-Madison
Madison, Wisconsin, USA
dkillough@wisc.edu

Justin Feng*
Department of Computer
Sciences
University of
Wisconsin-Madison
Madison, Wisconsin, USA
jfeng233@wisc.edu

Rithvik Dyava*
Department of Computer
Sciences
University of
Wisconsin-Madison
Madison, Wisconsin, USA
rdyava05@gmail.com

Zheng Xue “ZX”
Ching*
Department of Computer
Sciences
University of
Wisconsin-Madison
Madison, Wisconsin, USA
zzching@wisc.edu

Daniel Wang*
Department of Computer
Sciences
University of
Wisconsin-Madison
Madison, Wisconsin, USA
dwang448@wisc.edu

Yapeng Tian
Department of Computer
Science
The University of Texas at
Dallas
Richardson, Texas, USA
yapeng.tian@utdallas.edu

Yuhang Zhao
Department of Computer
Sciences
University of
Wisconsin-Madison
Madison, Wisconsin, USA
yuhang.zhao@cs.wisc.edu

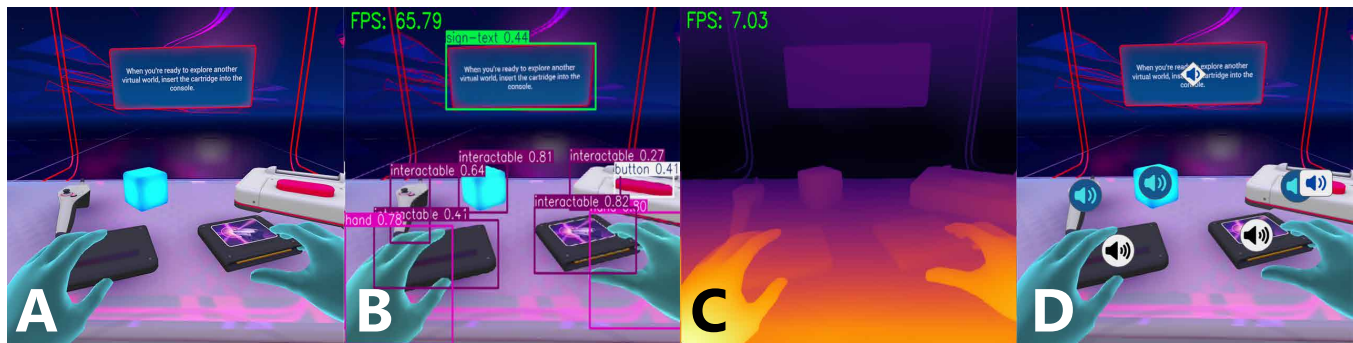


Figure 1: VRSight’s pipeline from (A) the original input frame, to (B) the fine-tuned object detection output, (C) depth map, and finally (D) spatial audio representation.

Abstract

Virtual Reality (VR) technologies are becoming increasingly mainstream. However, VR headsets largely rely on visuals to convey information, making them inaccessible to blind and low vision people (BLV). Therefore, we present VRSight, an end-to-end system using state-of-the-art machine learning models to empower BLV to more effectively interact with VR environments. VRSight applies to any VR application post-hoc, not needing developer implementation. VRSight uses our fine-tuned object detection model trained

on VR scenes to recognize virtual elements in each frame, like interactable objects, signs, and seating areas. VRSight automatically sonifies important objects like safety information while including a keyboard-based menu system for users to query other objects. All objects are sonified corresponding to their spatial locations in VR with sound effects and text-to-speech in an application-relevant tone. VRSight increases VR accessibility for BLV, serving as a foundation for more inclusive VR design and encourages broader participation in virtual social spaces.

* Authors 2-5 contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1395-8/25/04

<https://doi.org/10.1145/3706599.3721194>

CCS Concepts

• **Computing methodologies** → **Object detection**; *Scene understanding*; • **Human-centered computing** → **Accessibility systems and tools**; *Virtual reality*; *Interactive systems and tools*.

Keywords

Virtual Reality, Artificial Intelligence, Accessibility, Blind and Low Vision, Computer Vision

ACM Reference Format:

Daniel Killough, Justin Feng, Rithvik Dyava, Zheng Xue “ZX” Ching, Daniel Wang, Yapeng Tian, and Yuhang Zhao. 2025. Demonstration of VRSight: AI-Driven Real-Time Descriptions to Enhance VR Accessibility for Blind People. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3706599.3721194>

1 INTRODUCTION

Virtual Reality (VR) platforms enable diverse immersive experiences, and rapidly increasing hardware capabilities and computational power are causing VR to rapidly evolve from a niche technology to a mainstream tool for gaming [4], education [6, 8], health-care [16], and socializing [14, 26]. Despite major advances in VR hardware and software, VR accessibility (a11y) for BLV people has not kept pace. VR devices like the Meta Quest and Apple Vision Pro currently rely almost exclusively on visual prompts such as on-screen text, visual gestures, and color-coded objects to denote interactivity, all of which create significant barriers for blind and low vision people (BLV) [20, 21, 23]. Standardized visual a11y features also do not yet exist besides basic adjustments like color filters and text size [2]. A11y features for users needing support outside of colorblindness or magnification has yet to be fully introduced (e.g., for blind users, people with central vision loss, people with blurred vision), much less a 3D screen-reader, and developers are largely resistant to supporting them [9]. Some a11y research for 3D virtual environments has progressed for common BLV difficulties like in-scene navigation [17] and movement [18], but research lacks an end-to-end description system to describe virtual objects like a 3D screenreader. Furthermore, existing VR research largely requires developer intervention in a controlled environment, but users need a post-hoc solution that works in already-released applications [9].

Recent advancements in artificial intelligence (AI) research for object recognition, scene understanding, and depth sensing suggest promising uses and integrations of these technologies to allow a11y features for VR [5, 25, 27], with some tools used in a11y research for Augmented Reality (AR) and Mixed Reality (MR) devices to magnify text [19] and help BLV interact with touchscreens [10]. Both of these systems require real-time processing on visual data from a user’s real-world environment and generate augmentations much like a screen reading system. Research has investigated using computer vision methods like object detection to describe real-world environments, with datasets created to show real-world scenarios like traffic [15], sports [11], and cooking [12], but we have found existing models are not applicable to VR virtual environments—potentially due to the unique appearances of these items (even for VR objects also present in real-world datasets). VRSight offers a comprehensive dataset of VR objects alongside a corresponding fine-tuned object detection model to further identify VR environments.

In this work, we present VRSight, an end-to-end automated object detection system that leverages state-of-the-art machine learning models to interpret and translate real-time spatial information in virtual worlds through 3D audio. VRSight is capable of detecting over 30 categories of VR objects, working post-hoc in the background of any app running on the system. VRSight automatically identifies virtual objects based on various object classes

(e.g., interactables, avatars, seats, signs, guardian); processes the detected visuals to generate audio effects; combines the data with a depth map to spatialize the generated audio; and relays information back to users. VRSight also detects the scene’s tone and customizes the audio descriptions accordingly to enhance user immersion.

This demonstration of VRSight contributes the following:

- A dataset including 16,905 labeled VR images focused on a variety of top selling, free social VR apps.
- A fine-tuned, state-of-the-art object detection model trained on the custom dataset, supporting real-time virtual object detection for 33 VR object types.
- An end-to-end system combining multiple AI models to synthesize tone-relevant spatial audio from visual framedata. Our system works post-hoc on any VR application without needing developer intervention.

2 DESIGN OF VRSIGHT

VRSight is a post-hoc software tool that captures visual framedata from a VR application in real-time, processes the frames through a fine-tuned machine learning object detection model (YOLOv8 [7]) and a zero-shot depth estimation model (Depth Anything V2 [24]) to extract the corresponding spatial positions of different objects. These spatial coordinates and object information is processed and conveyed back to the user using sound effects and text-to-speech (TTS). This way, users can hear and understand VR environments alongside existing VR audio at any time.

VRSight was designed to address key problems in the CHI community, extending to broader themes of Accessibility, Interacting with Devices, Games & Play, and User Experience as they relate to improving human experiences using computer technology. Translating visual data into audio while maintaining VR’s existing haptics enable us to convey complex, immersive visuals to BLV people. VRSight is enabled by recent advances in AI, ultimately impacting how BLV may interact with all VR users while making VR more inclusive for all. We now offer the following insight into our dataset development and user flow procedures for our demonstration:

2.1 Dataset Development and Model Performance

VRSight consists of a multitude of systems working in conjunction, but VRSight’s primary, on-device systems consist of 2 models: Our fine-tuned YOLOv8 [7] and Depth Anything V2-Large [24]. To detect different virtual environments in our VR scenes, we first investigated existing datasets for an object detection model. Research has investigated using computer vision tools like object detection to describe real-world environments [15], but research lacks a VR-specific dataset for virtual objects, and our testing found that even skeumorphic objects like televisions or benches were inconsistently or seldom identified with existing models (e.g., COCO [13])—therefore, we needed to create our own:

We decided to focus our efforts on the most popular consumer VR headset line (Meta Quest) and VR apps, deciding to focus on social VR for its wide diversity in worlds, types of objects, and low barrier of entry. To that end we conducted an exhaustive search on the Meta Quest store for top selling applications related to free apps socializing with other users in virtual reality. Our search returned

24 free social VR apps, which we further narrowed to omit gambling (e.g., PokerStars), apps that are strictly games (e.g., Gorilla Tag), 2D apps (e.g., Facebook), apps with primarily real-life visuals (e.g., Woorld), and those requiring subscriptions after a free trial period (e.g., Villa, Immerse, Innerworld). Our final list included 15 social VR apps including VRChat, Rec Room, ROBLOX, Remio, Half + Half, Flipside, Alcove, Engage, Spatial, Meta Horizon Worlds, Zoe, vTime XR, MeetinVR, Multiverse, and Arthur. We decided to additionally include two official demos, Oculus First Steps and Oculus First Contact, to support apps that automatically open on setup for older VR devices like the Oculus Rift and Oculus Quest 1. These apps also contain controlled set of VR objects and we believe serve as a great demonstration of VR’s capabilities to new users. We plan to demonstrate VRSight using the first few minutes of Oculus First Steps to help users become acquainted with the headset.

Six researchers recorded VR footage using Quest 2, Quest Pro, and Quest 3 and uploaded still frames to Roboflow [3], a popular online computer vision labeling tool. We split these between 70/20/10 Train/Val/Test. We generated 3 augmentations for each training image with random variations of mirrored, rotated, cropped in up to 20%, and sheared up to 15° using Roboflow’s Generate Image Augmentations tool [1]. Our dataset consists of 16,905 images. We randomly excluded three apps from training (Engage, Spatial, and Half+Half), to verify the generalizability of our model, including them only in validation and testing with a 67/33 split. We fine-tuned a YOLOv8n [7] model on an NVIDIA A100 GPU for 250 epochs. (YOLOv8 appeared to have the highest mAP from our testing while demonstrating real-time latency.) Our final model evaluated at an mAP@50 of 65.3 and an mAP@50-95 of 47.7. (Compare to the base YOLOv8n trained on COCO, which reports an mAP@50-95 of 37.3 [22]). Running the model on an NVIDIA RTX 4090 GPU results in an average framerate of 96FPS (10.41ms per frame) for real-time prediction, indicating both high accuracy and speed.

For depth estimation, although we considered traditional computer vision approaches by triangulating the disparity between the VR headset’s lenses, possible distortion in Meta’s display mirroring utility produced invalid output. We therefore sought a monocular approach, and Depth Anything V2 [24] produced very good results for our use case. Running both models in parallel using Python’s Thread class on an NVIDIA RTX 4090 produces a frame every 21.54ms for the object model and 148.8ms for the depth model, which we believe to provide real-time feedback.

2.2 System Flow

2.2.1 System Requirements. VRSight is primarily contained within a single python script run on a companion PC. VRSight simply requires a VR headset and a connected computer with a capable-enough GPU to run the models. PCVR headsets (e.g., Pimax Crystal Light) are compatible but require higher-performance cards or a second GPU. Although we report our benchmark numbers using an RTX 4090, we have successfully run VRSight on a gaming laptop (Alienware M15) with an onboard RTX 3070 mobile GPU and achieved real-time performance. Since many users purchase standalone VR headsets due to their lesser overall cost and portability, we also focus on standalone VR devices for these metrics.

2.2.2 System Components. VRSight consists of the following primary components:

- **A Fine-Tuned Object Detection model (YOLOv8[7])** identifies 33 classes of VR-specific objects (e.g., avatars, buttons, signs) with bounding boxes and confidence scores.
- **A Zero-Shot Depth Estimation Model (DepthAnythingV2 [24])** generates depth maps of our VR scene to understand the spatial arrangement of objects. Using this map in conjunction with output from our object detection model helps accurately place spatial audio nodes in 3D.
- **A Multimodal Large-language Model (GPT-4o)** analyzes frames once per minute to set an appropriate tone for our TTS, and describes in-game graphical objects (e.g., posters) and iconography.
- **An Optical-Character Recognition (OCR) Model (Microsoft Azure AI Vision)** used on VR objects for classes containing mostly text. We pass its output to the TTS system:
- **A Text-to-Speech Recognition System (Microsoft Speech-Synthesizer)** generating text from GPT and OCR in an immersive tone. Played out through a WebVR utility running in the headset’s browser to convey directional audio cues.

We provide a flowchart of VRSight’s components in Figure 2.

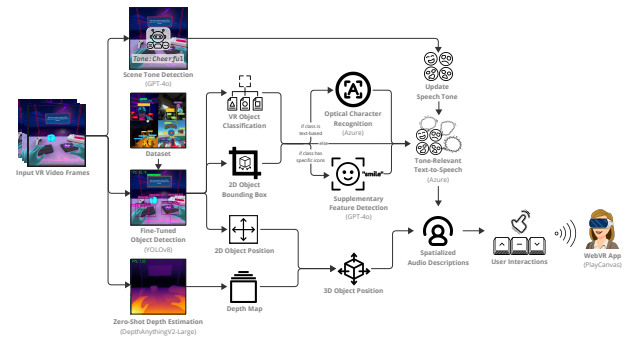


Figure 2: System flowchart showing a pipeline from the input VR video frames to the output webVR spatial audio.

Our GPT-4o, TTS, and OCR services all contact Microsoft Azure. Although contacting a cloud service adds latency, we found this approach more feasible with a single PC and granted higher accuracy than on-device computing methods, particularly for GPT and OCR. Furthermore, Azure contains many options for text-to-speech emotions to help us convey multiple scene tones to our users using the same base voice. Finally, our audio playback is enabled by Meta’s OS version 66 update, permitting background audio playback. This method seemingly does not restrict webVR interfaces from continuing an audio stream in the background, but it does restrict IMU access when not focused. Using a platform like PlayCanvas enables us add and remove objects in real-time, and changes automatically populate in the headset.

While the object detection model identifies a series of objects in each frame, the depth model preloads a map with the relative distance of each object in the virtual environment. We illustrate this system flow using an example VR image in Figure 3.

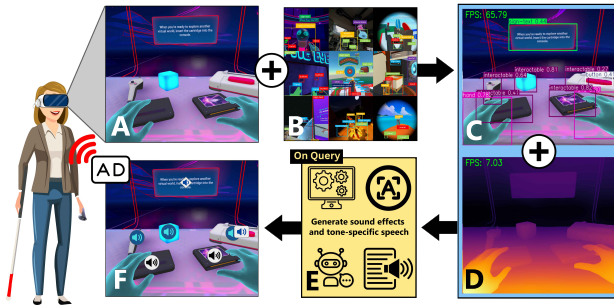


Figure 3: Technical system flow diagram. VRSight takes the input frame (A) from the virtual reality headset, where the object detection model (C) trained from dataset (B) identifies the scene’s VR objects. The depth estimation model (D) generates a depth map of the scene. The program (E) processes input from the models and keyboard, generating appropriate spatial audio cues (F). VRSight then sends the spatial audio descriptions wirelessly back to the VR headset.

To determine where to play audio, VRSight takes the midpoint of each detected object’s bounding box and queries the depth map to determine the object’s distance from the user. Based on the detected virtual objects, we then run GPT, OCR, or TTS as needed: VRSight uses GPT integration to describe heavily visual objects outside of our dataset, like framed photos inside the Rec Room tutorial’s art museum; OCR enables VRSight to accurately read classes with readable text embedded in the virtual environment, such as text-heavy signs, UI elements, labels, and avatar nametags; and our TTS engine then takes text input from the detected label, GPT, and OCR, and creates a base-64 encoded audio data object with the specified tone. Finally, we send packets containing a set of 3D location coordinates and an audio file if any TTS audio was synthesized to our webVR utility to be played in the VR headset.

2.3 Default User Interactions

VRSight works alongside the user’s VR headset on a separate computer, allowing seamless integration for any VR application and device with minimal additional setup. While VRSight’s detection system identifies virtual objects in real-time, playing all detected items for each frame may be overwhelming. Although VRSight plays some detected classes automatically, like important safety information to notify the user if they are existing their playspace, VRSight offers multiple default interaction methods for users to query their environment using a small 3-key keyboard we attached to one controller: Users can activate *SceneSweep* by pressing the top-most key on the keyboard closest to the VR controller’s grab button. *SceneSweep* audibly scans the entire VR environment horizontally from left to right, providing spatial audio cues corresponding to each detected object’s location and a categorization of the type of object (e.g., “sign”), without running any further OCR. The feature helps users navigate the VR environment easily by simulating an imaginative model through sequential audio feedback.

For full-scene descriptions, users press the middle key to activate *ContextCompass*, querying GPT to describe the current view as a

concise, contextual summary. This query is intended to help users orient themselves in the VR scene, (e.g., after teleporting).

Since VR applications commonly use pointers to interact with the environment, users press both the top and middle keys simultaneously to query objects near the current pointer position. Doing so also triggers *AimAssist*, which reads elements nearby the pointer without needing to point directly at each item. If the user executes this combined button gesture while the pointer is over a menu, *MenuPilot* takes over, reading out precise elements present around the pointer. Combined with tactile feedback through the VR controllers when hovering over interactable menu elements, VRSight may effectively communicate menu information to users. Should pointers not be available, VRSight defaults to reading out objects near the user’s hand or controller.

Finally, to aid user safety, VRSight incorporates *GuardianAlert*, a proactive feature designed to keep users from moving too far away from their original playspace. This feature automatically runs on every frame and issues an auditory warning prompting users to back up.

3 INITIAL FEEDBACK

We conducted a preliminary evaluation of VRSight with a pilot study consisting of three blind and low vision (one blind, two low vision) participants, who gave insightful feedback on the demonstration. All three participants were first-time VR users and needed a short tutorial on how to wear the headset and hold the controllers. Participants were very interested in the system and were excited for future direction. We asked participants what kinds of things they find difficult in social settings, and they notably mentioned trouble finding an open place to sit without a guide. While social VR allows people to sit nearly everywhere, we still included a dedicated seat-finding option in our menu with a filter for only vacant seats. In the future we plan to conduct a full user evaluation with blind and low vision community members and explore running VRSight in tandem with real-world object detection models (e.g., YOLOv8x on COCO) to further increase VRSight’s capabilities with more realistic objects.

4 FUTURE DIRECTION

In this paper, we present VRSight, a system enabling BLV people to explore visual objects in VR using a AI-driven audio descriptions. The development of VRSight introduces a revolutionary approach to a longstanding challenge in VR; by integrating advanced object recognition, immersive tone-based audio outputs, and intuitive user interactive capabilities, our system enables BLV to engage with VR environments previously unseen. VRSight helps to bridge the gap between modern technologies and a lack of existing developer support, and we aim to motivate continued VR a11y system development for users and developers alike. Although VRSight comes equipped with a default set of interactions to query the system (§2.3), we invite developers and researchers to devise novel interaction techniques that expand on these defaults for future work on accessible, intuitive interactions for VRSight. We look forward to seeing what you make.

Acknowledgments

This work was partially supported by the National Science Foundation under Grant No. IIS-2328182. Thank you to Yuheng Wu for helping select and troubleshoot object detection models and to Abhinav Nandwani for assistance annotating part of the dataset.

References

- [1] Roboflow 2023. *Create Augmented Images*. Roboflow. <https://docs.roboflow.com/datasets/image-augmentation>
- [2] Meta. 2024. *Learn about accessibility features for Meta Quest*. Meta. <https://www.meta.com/help/quest/articles/in-vr-experiences/oculus-features/accessibility-features/>
- [3] Roboflow 2025. *Roboflow: Build and Deploy Computer Vision Applications*. Roboflow. <https://roboflow.com/> Used by over 1 million engineers to create datasets, train models, and deploy to production..
- [4] Mr Ninad Janardan Dani. 2019. Impact of virtual reality on gaming. *Virtual Reality* 6, 12 (2019), 2033–2036.
- [5] Karan Desai and Justin Johnson. 2021. VirTex: Learning Visual Representations From Textual Annotations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, USA, 11162–11173.
- [6] Elliot Hu-Au and Joey J Lee. 2017. Virtual reality in education: a tool for learning in the experience age. *International Journal of Innovation in Education* 4, 4 (2017), 215–226.
- [7] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. 2023. *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>
- [8] Sam Kavanagh, Andrew Luxton-Reilly, Burkhard Wuenche, and Beryl Plimmer. 2017. A systematic review of virtual reality in education. *Themes in science and technology education* 10, 2 (2017), 85–119.
- [9] Daniel Killough, Tiger F Ji, Kexin Zhang, Yaxin Hu, Yu Huang, Ruofei Du, and Yuhang Zhao. 2024. XR for All: Understanding Developer Perspectives on Accessibility Integration in Extended Reality. *arXiv preprint arXiv:2412.16321* (2024).
- [10] Florian Lang and Tonja Machulla. 2021. Pressing a button you cannot see: evaluating visual designs to assist persons with low vision through augmented reality. In *Proceedings of the 27th ACM Symposium on Virtual Reality Software and Technology*. 1–10.
- [11] Jaewook Lee, Yang Li, Dylan Bunarto, Eujan Lee, Olivia H Wang, Adrian Rodriguez, Yuhang Zhao, Yapeng Tian, and Jon E Froehlich. 2024. Towards AI-Powered AR for Enhancing Sports Playability for People with Low Vision: An Exploration of ARSports. In *2024 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*. IEEE, 228–233.
- [12] Jaewook Lee, Andrew D Tjahjadi, Jiho Kim, Junpu Yu, Minji Park, Jiawen Zhang, Jon E Froehlich, Yapeng Tian, and Yuhang Zhao. 2024. CookAR: Affordance Augmentations in Wearable AR to Support Kitchen Tool Interactions for People with Low Vision. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–16.
- [13] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. 2014. Microsoft coco: Common objects in context. In *Computer vision—ECCV 2014: 13th European conference, zurich, Switzerland, September 6–12, 2014, proceedings, part v* 13. Springer, 740–755.
- [14] Divine Maloney and Guo Freeman. 2020. Falling asleep together: What makes activities in social virtual reality meaningful to users. In *Proceedings of the annual symposium on computer-human interaction in play*. 510–521.
- [15] Venkata Naresh Mandhala, Debnath Bhattacharyya, Vamsi B., and Thirupathi Rao N. 2020. Object Detection Using Machine Learning for Visually Impaired People. In *International Journal of Current Research and Review*. 157–166.
- [16] Fabrizio Mantovani, Gianluca Castelnovo, Andrea Gaggioli, and Giuseppe Riva. 2003. Virtual reality training for health-care professionals. *CyberPsychology & Behavior* 6, 4 (2003), 389–395.
- [17] Vishnu Nair, Jay L Karp, Samuel Silverman, Mohar Kalra, Hollis Lehm, Faizan Jamil, and Brian A Smith. 2021. Navstick: Making video games blind-accessible via the ability to look around. In *The 34th Annual ACM Symposium on User Interface Software and Technology*. 538–551.
- [18] Renato Alexandre Ribeiro, Inês Gonçalves, Manuel Piçarra, Leticia Seixas Pereira, Carlos Duarte, André Rodrigues, and João Guerreiro. 2024. Investigating Virtual Reality Locomotion Techniques with Blind People. (2024).
- [19] Lee Stearns, Leah Findlater, and Jon E Froehlich. 2018. Design of an augmented reality magnification aid for low vision users. In *Proceedings of the 20th international ACM SIGACCESS conference on computers and accessibility*. 28–39.
- [20] Mauro Teófilo, Vicente F Lucena, Josiane Nascimento, Taynah Miyagawa, and Francimar Maciel. 2018. Evaluating accessibility features designed for virtual reality context. In *Consumer Electronics (ICCE), 2018 IEEE International Conference on*. IEEE, 1–6.
- [21] Shari Trewin, Mark Laff, Vicki Hanson, and Anna Cavender. 2009. Exploring visual and motor accessibility in navigating a virtual world. *ACM Transactions on Accessible Computing (TACCESS)* 2, 2 (2009), 11.
- [22] Ultralytics. 2024. *Ultralytics YOLOv8 Performance Metrics*. Ultralytics. <https://docs.ultralytics.com/models/yolov8/#performance-metrics>
- [23] Alice Wong, Hannah Gillis, and Ben Peck. 2018. VR Accessibility Survey: Survey for People with Disabilities. <https://drive.google.com/file/d/0B0VwTVwReMqLMFLdzVVaVdaTFk/view>. Last accessed 21 December 2018.
- [24] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. 2024. Depth Anything V2. *arXiv:2406.09414* (2024).
- [25] Hongchan Yoon, Baek-Hyun Kim, Mukhiddinov Mukhridin, and Jinsoo Cho. 2018. Salient region extraction based on global contrast enhancement and saliency cut for image information recognition of the visually impaired. *KSII Transactions on Internet and Information Systems (TIIS)* 12, 5 (2018), 2287–2312.
- [26] Kexin Zhang, Elmira Deldari, Yaxing Yao, and Yuhang Zhao. 2023. A diary study in social virtual reality: Impact of avatars with disability signifiers on the social experiences of people with disabilities. In *Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility*. 1–17.
- [27] Youcai Zhang, Xinyu Huang, Jinyu Ma, Zhaoyang Li, Zhaochuan Luo, Yanchun Xie, Yuzhuo Qin, Tong Luo, Yaqian Li, Shilong Liu, Yandong Guo, and Lei Zhang. 2023. Recognize Anything: A Strong Image Tagging Model. *arXiv:2306.03514 [cs.CV]* (2023). <https://arxiv.org/abs/2306.03514>